

Spring Boot API Notes

Introduction

This note explains how to return objects and collections from Spring Boot APIs, and how to pass values using URL parameters.

Spring Boot controllers typically return JSON by default when using `@RestController`. The returned object is automatically serialized by Jackson.

Tip: Use `@RestController` instead of `@Controller` when building REST APIs.

1. Return an Object from API

1.1 Why use a separate class?

A simple DTO (Data Transfer Object) or model class makes your API responses structured and reusable.

1.2 Best practice

- Use PascalCase for class names: `Student`
- Put reusable models in a package such as `com.training.model` or `com.training.component`
- Use `@Component` only when the class needs to be managed by Spring as a bean

1.3 Example model class

```
package com.training.model;

import org.springframework.stereotype.Component;

@Component
public class Student {
    private int rollno;
    private int fees;
    private String name;
    private String course;

    public Student() {}

    public Student(int rollno, int fees, String name, String course) {
        this.rollno = rollno;
        this.fees = fees;
        this.name = name;
    }
}
```

```

        this.course = course;
    }

    public int getRollno() {
        return rollno;
    }

    public void setRollno(int rollno) {
        this.rollno = rollno;
    }

    public int getFees() {
        return fees;
    }

    public void setFees(int fees) {
        this.fees = fees;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}

```

1.4 Example controller method

```

@RestController
public class StudentController {

    @GetMapping("/stuinfo")
    public Student stuinfo() {
        return new Student(101, 120000, "Anuj Kapoor", "B.Tech.");
    }
}

```

1.5 How it works

- Spring Boot sees the returned `Student` object
- Jackson converts it into JSON
- The API response looks like:

```
{
  "rollno": 101,
  "fees": 120000,
  "name": "Anuj Kapoor",
  "course": "B.Tech."
}
```

1.6 Call the endpoint

`http://localhost:8888/stuinfo`

2. Return a List of Objects

APIs often return multiple records as a list.

Example

```
@GetMapping("/multirec")
public List<Student> multirec() {
    return Arrays.asList(
        new Student(101, 10000, "Manoj Kumar", "B.Tech."),
        new Student(102, 20000, "Ashok Kumar", "M.Tech."),
        new Student(103, 30000, "Vijay Kumar", "M.Tech."),
        new Student(104, 10000, "Alok Kumar", "B.Tech."),
        new Student(105, 120000, "Raj Kumar", "M.Tech.")
    );
}
```

Resulting JSON

```
[
  { "rollno": 101, "fees": 10000, "name": "Manoj Kumar", "course": "B.Tech." },
  { "rollno": 102, "fees": 20000, "name": "Ashok Kumar", "course": "M.Tech." },
  ...
]
```

3. Passing values through URL

3.1 Path variables

Use `@PathVariable` when values are part of the path.

Syntax

```
@GetMapping("/api/{var1}/{var2}")
public String apiName(@PathVariable String var1, @PathVariable String var2) {
    // ...
}
```

Example

```
@GetMapping("/whomi/{name}/{address}")
public String whomi(@PathVariable String name, @PathVariable String address) {
    return "Name=" + name + "<br>Address=" + address;
}
```

Example URL

`http://localhost:8888/whomi/Dhananjay/Naini`

3.2 Query string parameters

Use `@RequestParam` when parameters are optional or passed as key/value pairs.

Example

```
@GetMapping("/search")
public String search(
    @RequestParam String name,
    @RequestParam(required = false, defaultValue = "unknown") String city) {
    return "Name=" + name + "<br>City=" + city;
}
```

Example URL

`http://localhost:8888/search?name=Rani&city=Lucknow`

3.3 When to use which

- Use `@PathVariable` for required values that identify a resource
- Use `@RequestParam` for filters, search terms, pagination, and optional inputs

4. Good API design tips

- Keep class names capitalized: **Student**
 - Use DTO/model classes for response data
 - Prefer **@RestController** for REST APIs
 - Return **List<T>** for collections
 - Validate input when accepting URL parameters
 - Use meaningful endpoint names
-

5. Practice problem

Create an API named `/empinfo` that returns employee data with fields: - `empid`
- `ename` - `edesig` - `esal`

Sample endpoint:

```
@GetMapping("/empinfo")
public Employee empinfo() {
    return new Employee(1, "Sonal", "Developer", 45000);
}
```